

BoilerRoom - FocusTrack

FocusTrack, är uppbyggd som en klassisk React-applikation där hooks används för att knyta samman tre centrala delar: lokalt state i komponenter, globala inställningar som språk och tema, samt synkronisering med data i localStorage. Detta innebär att vissa komponenter fokuserar på presentation, medan andra hanterar logik, datainhämtning och interaktion med webbläsaren. Uppdelningen gör applikationen mer strukturerad och lättare att vidareutveckla.

Om appen

Flödet startar i **main.jsx**, där applikationen bootstrappar hela appen. Här sätts **BrowserRouter** och olika providers upp innan **<App />** renderas. Själva hook-logiken för språk ligger däremot inte i **main.jsx**, utan är inkapslad i LanguageProvider i **useLanguage.tsx**. Där samverkar **useState**, **useEffect**, **useMemo** och **useContext** för att lagra språkval, spara det i localStorage, stabilisera context-värdet och göra översättningarna tillgängliga i hela komponentträdet.

Routing och layout hanteras i **App.jsx** och **AppLayout.jsx**. App definierar vilka sidor som kopplas till olika URL:er, medan AppLayout anpassar gränssnittet efter enhet. Med hjälp av hooks kan appen avgöra om användaren är på mobil eller desktop och växla layouten därefter. Utöver webbversionen har vi även en timer som kan köras i **Expo Go**, vilket gör att timern kan testas och användas i mobilformat.

Dashboarden fungerar som navet i applikationen och samlar funktioner som *timer*, *planering*, *statistik*, *mål* och *senaste sessioner*. Därifrån kan användaren snabbt navigera vidare till att starta en fokussession, se tidigare sessioner, planera sitt arbete, följa sin statistik eller ändra inställningar. Dashboarden kopplar därmed ihop hela appens innehåll och gör det enkelt att nå alla delar.

Appen använder också ett väder-API för att hämta aktuell väderinformation. Det gör att användaren kan se vädret direkt i appen, vilket ger en mer levande och användbar översikt i dashboarden. Väderfunktionen är byggd med React-hooks som **useEffect** och **useState**, där API-anropet görs direkt i väderkomponenten för att hämta aktuell väderdata och användarens position.

En central del av appen är mål och streaks. I Settings kan användaren sätta upp dagliga och veckovisa mål, och i statistikvyn visas hur långt användaren har kommit mot sina mål. Där visas också både nuvarande streak och bästa streak, vilket gör funktionen viktig för att följa kontinuitet och motivation över tid. En annan viktig funktion är dataöverföring. Appen har stöd för export och import av sessionsdata i **JSON-format**. Vid import kan användaren välja mellan två lägen: replace, som ersätter befintlig data, och merge, som slår ihop importerad data med den data som redan finns. Detta gör det möjligt att både säkerhetskopiera och återställa användarens historik på ett flexibelt sätt.

Den mest avancerade logiken finns i timerdelen. I **TimerSection.jsx** används flera **useState** för att hantera timerns tillstånd, som starttid, timerläge, sessionens etikett och inställningar. **useRef** används för värden som inte ska orsaka omrendering, till exempel ljudobjektet för alarm. **useEffect** hanterar bieffekter som intervall, lagring i localStorage och kontroll av nedräkning, medan **useMemo** används för beräkningar som återstående tid och progress.

Timer-logiken har också förbättrats med ett tydligare alarmflöde, där användaren kan stänga av alarmet via en mute-toggle (`timerAlarmMuted`) och där ett time-up prompt/jingle-flöde hanterar vad som händer när tiden är slut. Timern är dessutom kopplad till planeringen. Det handlar alltså inte bara om en separat Planning-sida, utan om att användaren kan använda dagens plan direkt i timern via funktionen *“Use this plan”*. När en session sedan sparas kan den planerade aktiviteten också markeras som klar. På så sätt binds planering och genomförande ihop i samma arbetsflöde.

Timer.tsx är en presentationskomponent som visar timern visuellt. Den ansvarar inte för logiken bakom funktionerna, utan fokuserar på rendering och animationer utifrån de värden den får. **StartSessionModal.jsx** används vid skapande och redigering av sessioner. Den hanterar local state, UI-logik och kopplingar till planering, historik och språk. Ett viktigt tekniskt beslut här var att rendera modalen via **createPortal**, så att den inte klipps av layoutens kort eller påverkas av overflow och stacking context. Detta var ett konkret exempel på en teknisk utmaning där lösningen förbättrade både struktur och användarupplevelse.

Sessionshanteringen samlas i custom hooken **useSessions.js**, som gör sessionsdata reaktivt. Den används i komponenter som `SessionList` och `RecentSessions`, där hooks styr visning och filtrering. Statistikfunktionerna bygger vidare på detta genom **useStatsData.js**, som beräknar exempelvis total tid, antal sessioner, målprocess och streaks och sedan används i presentationskomponenter. **Planning.jsx** använder hooks för att hantera formulär, filter, datum och drag-and-drop, samt synkroniserar data med en gemensam store. Den är integrerad med andra delar av appen, särskilt timern, vilket gör planeringen till en aktiv del av användarens fokusflöde.

Övriga komponenter, som `ToDo` och `Weather`, följer samma mönster med **useState**, **useEffect** och **useMemo**. Temahantering sker via den återanvändbara hooken **useLocalStorage**, som används i flera delar av gränssnittet. Sammanfattningsvis används hooks konsekvent genom hela applikationen: **useState** för local state, **useEffect** för synkronisering, **useMemo** för beräkningar och **useRef** för stabila värden. Custom hooks samlar återanvändbar logik och bidrar till en tydlig, sammanhängande och skalbar struktur.

Användning av AI- verktyg i projektet

Vi har med hjälp av AI tagit fram vår README-fil, där innehållet bland annat beskriver installation och projektstruktur. Även om AI har varit ett stort stöd har vi själva granskat filen noggrant för att säkerställa att all information är korrekt innan den publiceras.

AI har också varit till stor hjälp när det gäller dokumentation och kommentarer i koden. Det har stöttat oss i att knyta ihop funktionerna och tydliggöra hur olika delar av applikationen hänger samman i dokumentationen. Kommentarer i koden fungerar dessutom som ett stöd för att förstå vad olika kodrader, funktioner, typer och moduler gör.

Utmaningar i projektet

En av de större utmaningarna var att spara användardata och samtidigt använda den för att skapa relevanta rekommendationer. Lösningen blev att dela in sessionstider i olika intervaller, till exempel 5, 10, 15, 20 och 25 minuter. Genom att analysera användarens historik inom dessa intervaller kan systemet identifiera vilken tidslängd som passar bäst och

föreslå den vid nästa session. För att öka tillförlitligheten infördes även en "confidence"-funktion som visar hur säker rekommendationen är. Den baseras på antal genomförda sessioner samt användarens betyg av dem.

En annan utmaning var att arbeta i varandras kod under projektets slutskede. Att sätta sig in i någon annans struktur och logik var svårt, särskilt när nya funktioner som språkstöd, planeringskoppling till timern och förbättrad modalhantering skulle läggas till. Det blev en viktig lärdom i att utveckling inte bara handlar om att skriva egen kod, utan också om att kunna läsa, förstå och vidareutveckla någon annans lösning.

Begränsningar och framtida arbete

Appen är idag byggd som en local-first-lösning där data sparas i localStorage. Det innebär att användardata, inställningar, mål och historik lagras lokalt i webbläsaren. I nuläget finns ännu ingen backend, ingen autentisering och ingen molnsynkronisering. Ett naturligt nästa steg i framtida arbete vore därför att koppla appen till en backend-lösning med inloggning och synk mellan olika enheter.